CO-1 ASSESSMENT TOOL - 2

NAME:
REG NO:

1.

BANKING SYSTEM - ADT TO DATA STRUCTURES CONCEPT MAP

ADT (Abstract Data Type)

Account ADT

An abstract model for a Bank account that defines what operations can be performed, without specifying how they are implemented.

OPERATIONS (what it does)

Banking Operations

* `createAccount()` - create a new account
* `deposit(amount)` - add money to account
* `withdraw(amount)` - deduct money from account
* `getBalance()` - Check account Balance
* `getTransactionHistory()` - View all transactions
* `closeAccount()` - close an account

INTERFACE (How it is exposed)

Account Interface (Public view)

Defines the set of operations available to the user/other modules

```
interface Account
{
  void createAccount();
  void deposit(double int);
  boolean withdraw(double int);
  double getBalance();
  List<Transaction>
  getTransactionsHistory();
  Void CloseAccount();
}
```

Clients use only this interface not the implementation

IMPLEMENTATION (How it works)

Account Implementation

Contains the actual code that performs operations defined in the interface.

```
class BankAccount implements
Account
{
  private string actNumber;
  private double balance;
  private list<Transaction>
  transactions;
  ... Other methods ...
}
```

Clients are unaware of how it is implemented

DATA STRUCTURES (what is used)

Hash Table / Map
(accNumber → Account)
Fast lookup of accounts by account number

Array / List
To store list of transactions

Linked List / Queue
To maintain transaction history in order

File / Database
Persistent storage of accounts and transactions.

How ADTs separate Logical Design from implementation

* The ADT (Account) defines WHAT operations are available (logical design)
* The Interface exposes those operations without revealing HOW they are implemented.
* The implementation decides HOW to perform those operations using Data structures.
* Thus changes in implementation or data structures do not affect the client code that uses the interface

WHY THIS ABSTRACTION IS BENEFICIAL

MODULARITY	REUSABILITY	MAINTAINABILITY
Each component (interface, implementation, data structures) is independent. Changes in one module don't affect others	The same ADT / interface can be reused in different systems (eg ATM, online Banking) with different implementations	Implementation or data structures can be optimized or replaced without changing the interface or client code

Operation	Example
Deposit	Deposit ₹10,000 into account 12345
Withdraw	Withdraw ₹2,000 from account 12345
Balance check	check balance of account 12345
Transaction History	View last 10 transactions
Account closure	Close account 12345

VIDEO STREAMING SERVICE - CONCEPT MAP

DATA ACCESS PATTERNS

How the system reads and writes watch history

Operations
- Record a video watched
- Update watch progress
- Read full watch history
- Get recent videos watched
- Search / jump to a specific video in history
- Delete an entry

Workload Characteristics
- More reads than writes
- Recent videos accessed frequently
- Occasional access to older items
- Insertions at the end (new watches)

FREQUENTLY READS AND UPDATES USER WATCH HISTORY

2A. SEQUENTIAL ACCESS

Operations that process data in order from start to end

Examples
- Display full watch history
- Show recent activity
- Export / backup history
- Clear all history

↓

Best suited for

When we need to traverse the history in order

2B. RANDOM ACCESS

Operations that jump directly to a specific item.

Examples
- Fetch details of a specific video in history
- Update progress of a particular video
- Remove a specific entry
- Check if a video exists in history

Best suited for

When we need fast lookup, update or delete by key

DATA STRUCTURES

For sequential access

Linked List / Dynamic Array
- Efficient traversal
- Easy to append new items
- Good for getting recent items in order

For Random access

Hash Map (Dictionary)
- O(1) average into time lookup
- Fast check for existence
- Easy data delete by key.

Hybrid (Recommended)

Use both for best performance:
- List (or deque / linked list) to maintain order of history
- Hash map to access videoID quickly

PERFORMANCE

Sequential structures
- Traversal : O(n)
- Insert at end : O(1)
- Memory Overhead : Low
- Best for showing history in order

Hash Map
- lookup : O(1) average
- update : O(1) average
- Delete : O(1) average
- Memory overhead : Higher
- Best for quick access to any item

Hybrid Approach (Best F.S)
- Fast ordered Traversal
- Fast random lookup / update
- Scales well with large history
- Balances memory & speed

HOW ACCESS PATTERS INFLUENCE STRUCTURE SELECTION.

- If operations are mostly sequential (viewing history, recent activity), choose structures optimized by traversal
- If operations require frequent jumping to specific items (update progress, fetch by video), choose structures optimized for direct access.
- Real systems often require both → choose a combination that matches the workload.

JUSTIFICATION OF MOST SUITABLE APPROACH

A hybrid approach (ordered list + Hash map) is most suitable for video streaming service because

⇒ users mostly view history in order → list is efficient

⇒ updates / lookups by videoID are frequent → HashMap is fast.

⇒ Supports both current and future features (search, delete, analytics) efficiently.

⇒ offers the best overall performance and user experience.

EXAMPLE FLOW

1. User watches a video → append to list + add to map
2. User update progress of a video → Find in Map (O(1)) and update
3. User opens history → Traverse list (O(n)) to display in order
4. System remains fast even with millions of records.